Treeline

Compile AI Decisions into the Network Hardware

OpenHackathon Final • A2-05 • WinLab, NYCU

Workload offload + AI-assisted development

OpenACC

More Science, Less Programming

A2-05 ｜孟勳老師剛剛說我們可以，邪萬你說呢

TreeLine: Line-Rate DDoS Detection via DPU Offload

Compile a Random Forest into the BlueField DOCA Flow hardware pipeline — per-packet ML classification at line rate, no CPU in the fast path

Team members

汶

蔣汶儒
Wen-Ju Chiang
NYCU · Team lead

洺

廖洺玄
Mike Liao
NYCU

彥

郭彥廷
Tim Kuo
NYCU

詠

詹詠翔
Loki
NYCU

昱

沈昱宏
Stanley
NYCU

Mentor & advisor

RH

Richard Huang
NVIDIA AI Technology Center (NVAITC) · Mentor

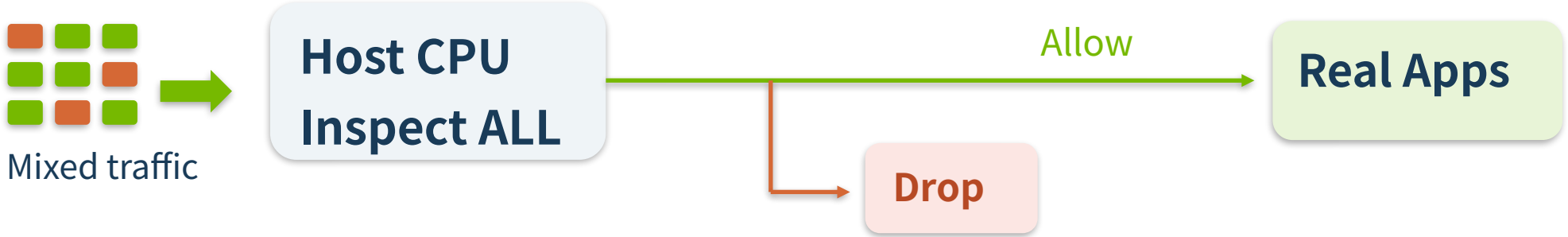
超

曾建超 老大
NYCU Computer Science · Advisor

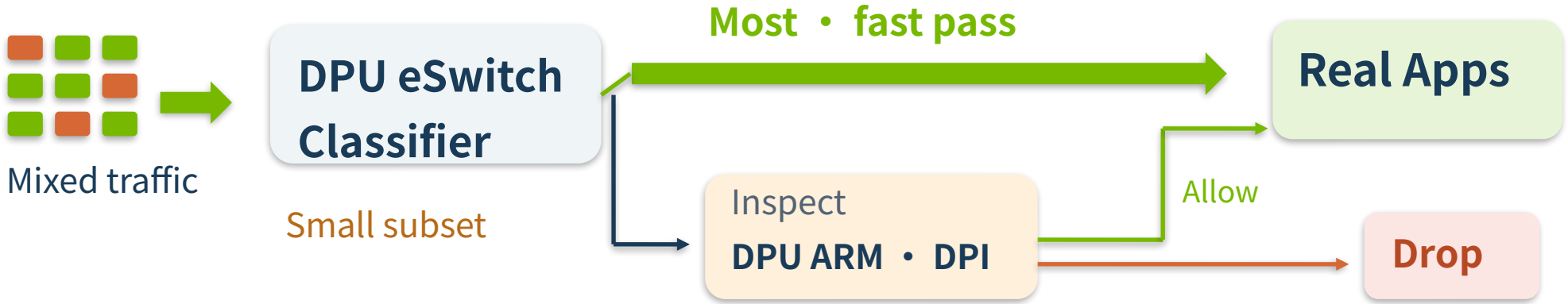
OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA

Inspect All on CPU. Selectively Inspect on DPU.

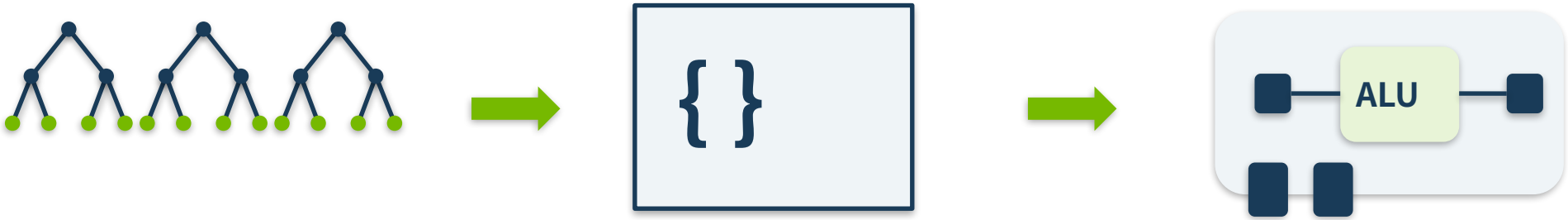
BEFORE • Every packet is inspected on the Host CPU



AFTER • DPU handles inspection; Host CPU runs applications



Let the Network Hardware Decide



Trained model

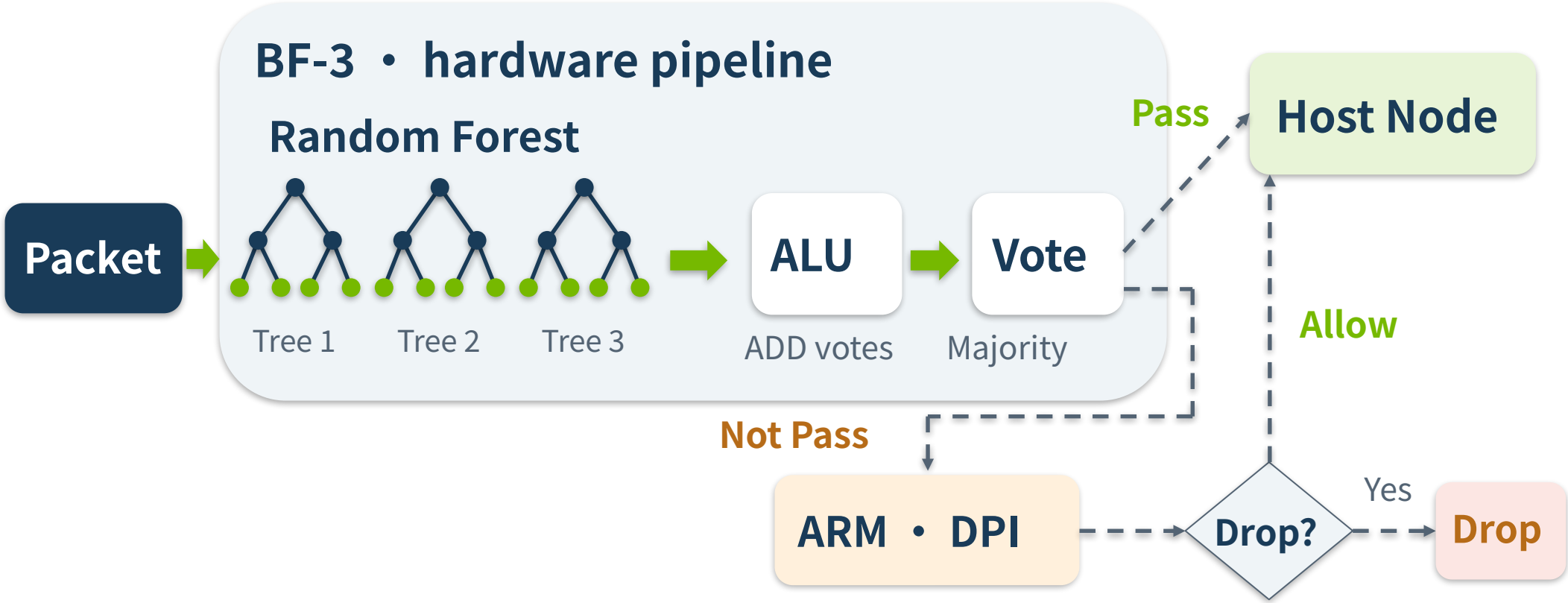
Treeline compiler

DPU

A trained model becomes executable hardware rules.

Random Forest First, DPI When Needed

TARGET ARCHITECTURE • Fast-path classification, selective deep inspection.



The Compiler Is the Bridge

TECHNICAL HIGHLIGHT • Turn learned logic into the operations the DPU actually supports.

Threshold	→	**Packet-field comparison**
Tree branch	→	**Pipe-to-pipe forwarding**
Forest votes	→	**ADD + final class decision**

Model → lowering → generated C → SDK build → installed rules

BF-3 ALU Enables In-Pipeline Voting

KEY ENABLER • ALU arithmetic accumulates votes inside the hardware pipeline.

Earlier design

Each tree writes its result
Combine the stored votes
State use grows with the design



BF-3 pipeline ALU

ADD each positive vote
Accumulate a running count
Reuse state across trees

4 scratch fields

A3 implementation

In-pipeline ADD voting.
A tested 33-tree forest.

CPU vs Treeline | Performance Comparison

64B REFERENCE BENCHMARK • Team-reported BF-2 results

METRIC	CPU	TREELINE	GAIN
Throughput (64 B)	12.4 Gbps	**44–47 Gbps**	3.6×
10M-packet time (derived)	≈411.5 ms	**≈115.6 ms**	3.6×
Host CPU inspection worker	≈100% (est.)	**0% (offload)**	Offloaded
Per-packet latency indicators	0.076 µs	**+2 µs added**	N/A

3.6× speedup with NO latency degradation

Building the DPU Path Takes Work

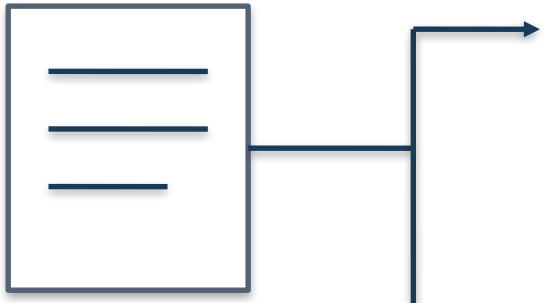
Find APIs → Code & debug → Book a slot



Late mistakes consume scarce hardware time.

Make Agent Support Useful

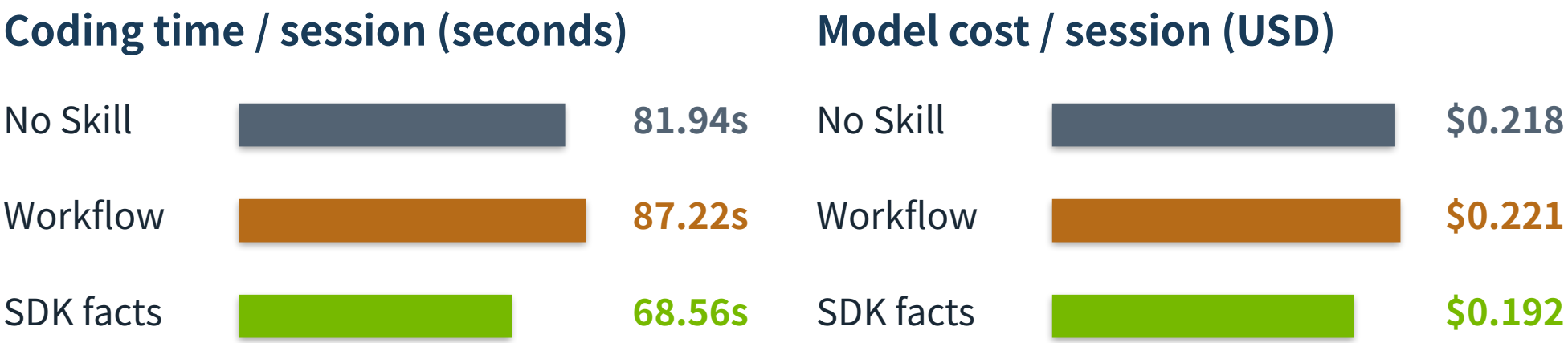
BEFORE	WHAT WE ADDED	TIME ↓	COST ↓
Generic notes	SDK → SDK facts	21.4%	13.4%
Generic notes	{ } → Code reuse	41.9%	36.9%



No benefit observed

85 coding sessions • 4 follow-up studies

No Skill vs Workflow Guide vs SDK Knowledge



SDK facts vs no Skill: −16.3% time • −12.0% cost

SDK facts vs old guide: −21.4% time • −13.4% cost

A Shorter Export Task with Reusable Code

WORKFLOW HIGHLIGHT • Same agent and task; five independent sessions per arm.

Mean coding time • SDK3.5 EXPORT



41.9%

Time saved

36.9%

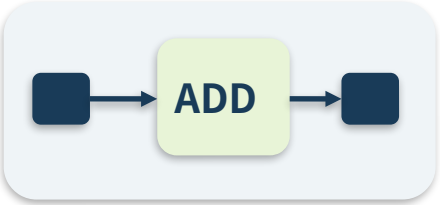
Cost saved

Model cost: $0.321 → $0.202 / session

Selected-suite passes: 5/5 per arm

Two Concrete Results. One Practical Next Step.

AI ON DPU



33

Trees • verified in hardware
ALU voting • depth 3

Golden: 1,449/1,449 per model

AI HELPING DEVELOPMENT

{ }

−41.9%

coding time

−36.9% model CLI cost

EXPORT task • reusable code assets

Thank You

Compile the decision.
Verify it on real packets.

A2-05 · WinLab, NYCU

Questions & Discussion

OpenACC
More Science, Less Programming

Energy Efficiency | Measure the Same Work

CPU vs Treeline • Same N correctly handled packets • Power and savings pending

$$E = P_{avg} \times t$$

$$W \times s = J$$

MEASURE	CPU	TREELINE
Average system power	[TBD W]	[TBD W]
Same-N completion time	[TBD s]	[TBD s]

$$\text{Energy savings} = 100 \times (1 - E_{Treeline} / E_{CPU}) = \text{TBD}\%$$

Wishlist

Tools

Version-matched SDK evidence
Offline build + fault checks

Language / standards

Typed field widths
Explicit action + lifetime contracts

Event

Ready-to-run exclusive slots
Visible ownership and readiness

Systems

Isolated traffic generator
Stable routes + BF-3 access

Final Thoughts

Worth it?

Yes: 33 trees on BF-3

1.72× faster EXPORT coding

Continue?

Yes: opt-in compiler integration

Reusable SDK checks and evidence

Next?

Matched CPU / BF-3 timing

Full DPI path + system energy

Support needed

Stable, exclusive DPU access

SDK expertise + power tools

Application Background

Normal traffic and attacks share the wire.

Compile tree decisions into the DPU.

Keep host resources for applications.

AI workflow | EXPORT



Mean coding time • five runs per arm

Hackathon Objectives and Approach

Model → compiler → DOCA Flow

BF-3 ALU accumulates votes in-pipeline.

Agent support: SDK knowledge + code reuse.

Technical Accomplishments and Impact

BF-3: 33 trees • 4 scratch fields

Golden match: 2 models × 1,449 lengths

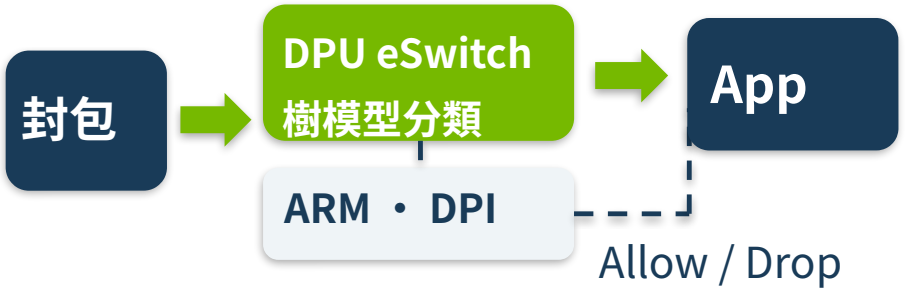
BF-2 reference: 3.6× reported at 64 B

EXPORT: 1.72× faster • CLI cost −36.9%

OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA

讓封包先在 DPU 判斷，把主機留給應用

目標路徑：先分類，必要時深入檢查



A2-05 • NYCU WinLab • 指導：曾建超

AI： 1.72×（EXPORT 單題）

DPU： 3.6×

AI coding time • 同 Agent、各五次

165.34 s → **96.09 s**

給一般讀者的說明

當正常流量與攻擊封包同時大量湧入，若全部交由主機檢查，就會與應用程式爭用 CPU 資源，可能影響服務效能。 Treeline 將決策樹模型轉換成專門處理網路資料的 DPU 可執行的規則，由硬體完成條件比對並彙整判斷結果，目標是讓多數封包通過初步檢查後直接送往應用程式，只有少數需要深入分析的封包，再交由 DPU 內的處理器處理。目前處理速度為單核 CPU 的 3.6 倍，且幾乎沒有效能的損失。此外，團隊也改進現有的 AI Skill 使得原本幾乎對 AI 沒有幫助的 Skill 轉化成可達平均執行時間由 165.34 秒縮短至 96.09 秒，且執行速度約為原本的 1.72 倍，模型命令列工具的使用費用降低 36.9% 。

Backup | Verification Behind the Highlights

3 NOVEMBER OFFLINE REVALIDATION • Supporting evidence, not new AI or hardware samples.

203 tests passed

91 compiler / modular
47 workflow tools
65 knowledge tools

627 SDK checks

EXPORT: 323 / 323
DEPLOY: 304 / 304
Compile / link + faults

12 SDK builds

10 algorithm configs
2 diagnostics
15,760 normal cases

New flat backend: 33,554,432 abstract inputs; zero mismatches.
Two fixed models; SDK-recorded graph, not physical packets.

Backup | Benchmark Definitions & Evidence

BF-2 reference ≠ new BF-3 measurement • Reported, derived and inferred values separated

64B reference

CPU: 24.3 Mpps

Treeline: 86.5 Mpps

Rate ratio: 3.6×

Sender-limited: reported

Derived / inferred

10M: 411.5 → 115.6 ms

CPU worker: ≈100% inferred

≈4 cores: linear estimate

Not timed / scaling-tested

Latency boundary

Mean RTT: 71 → 73 µs

Added mean RTT: +2 µs

CPU classify p99: 0.076 µs

Different scopes; ratio N/A